

PRÁCTICA 2: Tipo Abstracto de Dato Pila

OBJETIVOS

- Aprender a utilizar el TAD Pila para resolver problemas.
- Aprender a elegir la estructura de datos apropiada para implementar el TAD Pila.
- Codificar sus primitivas y utilizarlo en un programa principal.
- Aprender a utilizar una librería estática en C.

NORMAS

Los programas que se entreguen deben:

- Estar escritos en C, siguiendo las normas de programación establecidas.
- Compilar sin errores ni warnings incluyendo las banderas `-Wall` y `-pedantic` al compilar.
- Ejecutarse sin problema en una consola de comandos.
- Incorporar un adecuado control de errores. Es justificable que un programa no admita valores inadecuados, pero no que se comporte de forma anómala con dichos valores.
- No producir fugas de memoria al ejecutarse.

PLAN DE TRABAJO

- **Semana 1:** Ejercicio 1. Prueba de la biblioteca de Pila y Algoritmo de combinación de pilas.
- **Semana 2:** Ejercicio 2. Algoritmo de recorrido del grafo en profundidad.
- **Semana 3:** Ejercicio 3. Implementación y prueba de una biblioteca para el TAD Pila.

Cada profesor indicará en clase cómo realizar las **entregas parciales semanales**: papel, e-mail, Moodle, etc.

La entrega final se realizará a través de Moodle, siguiendo escrupulosamente las instrucciones indicadas en el enunciado referentes a la organización y nomenclatura de ficheros y proyectos. Se recuerda que el fichero comprimido que se debe entregar debe llamarse `Px_EDAT_Gy_Pz`, siendo `x` el número de la práctica, `y` el grupo de prácticas y `z` el número de pareja (ejemplo de entrega de la pareja 5 del grupo 2161: `P2_EDAT_G2161_P05.zip`).

El fichero comprimido debe contener la siguiente organización de ficheros:

```
--- P2_EDAT_Gy_Pz/  
|--- types.h  
|--- file_utils.h  
|--- vertex.c  
|--- vertex.h  
|--- graph.c  
|--- graph.h  
|--- p2_e1a.c
```

```
|--- p2_e1b.c  
|--- p2_e2.c  
|--- Makefile  
|--- libstack.a  
|--- stack.c  
|--- stack.h  
|--- g2.txt  
|--- grades1.txt  
|--- grades2.txt  
|--- cities1.txt  
|--- cities2.txt
```

Las fechas de subida a Moodle del fichero son las siguientes:

- Los alumnos de Evaluación Continua, el 20 de marzo hasta las 23.55 horas.
- Los alumnos de Evaluación Final, según lo especificado en la normativa.

Ejercicio 1.

Ejercicio 1a.

Se desea implementar un algoritmo que permita elaborar un ránking de las mejores notas de EDAT. Para ello, se dispone de las notas (números reales) de dos grupos de forma ordenada y se desea crear un ranking conjunto y ordenado con las notas de ambos.

Debes hacer uso de la librería del TAD Stack `libstack.a` y la interfaz pública definida en el archivo de cabecera `stack.h`.

Algoritmo

El siguiente algoritmo recibe dos pilas ordenadas de mayor a menor ($s1$ y $s2$) y genera una pila ordenada (s) de menor a mayor. En el pseudo-código no se ha considerado el oportuno control de errores.

Algorithm 1: Merge stacks

```
input  : Stack: s1
        Stack: s2
        Stack: s
output: Status: st
1 while stack_isEmpty (s1)  $\equiv$  False & stack_isEmpty (s2)  $\equiv$  False do
2   | if stack_top (s1) > stack_top (s2) then
3   |   e = stack_pop (s1)
4   | else
5   |   e = stack_pop (s2)
6   | end
7   | stack_push (s, e)
8 end
9 // Detect the non-empty stack
10 if stack_isEmpty (s1)  $\equiv$  True then
11   | ps = s2
12 else
13   | ps = s1
14 end
15 // Pop from the non-empty stack
16 while stack_isEmpty (ps)  $\equiv$  False do
17   | e = stack_pop (ps)
18   | stack_push (s, e)
19 end
20 return st
```

Codificación del algoritmo

Codifica el algoritmo anterior mediante la función

```
/**
 * @brief: Merges two stacks
 *
```

```
* @param sin1, first input stack
* @param sin2, second input stack
* @param sout, result stack
* @return The function returns OK or ERROR
**/
```

```
Status mergeStacks (Stack *sin1, Stack *sin2, Stack *sout);
```

La función deberá incluir el oportuno control de errores.

Comprobación de la corrección de la función

Crea un programa **p2_e1a.c** que utilice la función **mergeStacks** para elaborar el ranking ordenado con las notas de dos grupos. El programa recibirá dos argumentos de entrada, correspondientes al nombre de los dos ficheros con las notas de cada grupo. El formato de los ficheros será el siguiente: en la primera línea habrá un número entero, indicando cuántos elementos (notas) contiene el fichero. En las siguientes líneas, habrá tantas notas (ordenadas de menor a mayor) como se haya indicado anteriormente, separadas por un salto de línea. Cada nota es un número real con un decimal. Puedes encontrar los ficheros de prueba en Moodle (grades1.txt y grades2.txt).

Se deberá utilizar el TAD pila para la gestión de los rankings de notas y la creación del ranking conjunto. La ejecución del programa deberá comportarse según muestran los siguientes ejemplos:

```
$ ./p2_e1a grades1.txt grades2.txt
Ranking 0:
Size: 10
9.750000
9.500000
9.500000
9.200000
8.000000
7.300000
6.500000
5.000000
4.200000
2.500000
Ranking 1:
Size: 8
10.000000
9.300000
9.200000
9.000000
7.400000
5.800000
4.750000
1.000000
Joint Ranking:
Size: 18
1.000000
2.500000
```

```
4.200000
4.750000
5.000000
5.800000
6.500000
7.300000
7.400000
8.000000
9.000000
9.200000
9.200000
9.300000
9.500000
9.500000
9.750000
10.000000
```

El programa utilizará la biblioteca estática `libstack.a` para gestionar la pila. En el Anexo 2 puedes encontrar cómo crear y usar librerías estáticas en tu proyecto.

El programa deberá gestionar correctamente la memoria.

La función deberá usar llamadas a `stack_print` para mostrar el contenido de las pilas. Puedes encontrar un anexo sobre el uso de punteros a función Anexo 1.

Utiliza `valgrind` para verificar que el programa no tiene fugas de memoria.

Ejercicio 1b.

El ejercicio anterior presentaba una limitación y es que sólo funcionaba con pilas que contuviesen tipos de dato float. Sin embargo, C es lo suficientemente flexible para poder hacer implementaciones más genéricas. Esto lo podremos aprovechar gracias a los punteros a funciones. Para ello:

- Modifica el prototipo de la función `mergeStacks` para que reciba un puntero a función con la referencia a la función de comparación de los elementos de la pila.
- Modifica la función `mergeStacks` para que utilice el puntero a función recibido para comparar los dos elementos de las pilas
- Comprueba el funcionamiento del programa con la función de comparación de float facilitada en `file_utils.h`.

Comprobación de la corrección de la función

Crea un programa `p2_e1b.c` que utilice la función `mergeStacks` para elaborar el ranking ordenado de dos grupos de elementos de tipo vertex.

El programa recibirá dos argumentos de entrada, correspondientes al nombre de los dos ficheros, que deberán ser leídos y almacenados como tipos de dato vertex. El formato de estos ficheros es como en la práctica anterior: en la primera línea habrá un entero indicando el número de elementos que hay en el

fichero. En las siguientes líneas habrá una definición de vertex por línea. Puedes encontrar los ficheros de prueba en Moodle (cities1.txt y cities2.txt).

Se deberá utilizar el TAD Pila para la gestión de los grupos de vertex y la creación del grupo conjunto.

Modifica la llamada a la función `mergeStacks` para que reciba un argumento adicional con un puntero a la función de comparación de vértices.

Modifica la impresión de las pilas para que llame correctamente a la función de impresión de tipos de dato `vertex`.

La ejecución del programa deberá comportarse según muestran los siguientes ejemplos:

```
$ ./p2_e1b cities1.txt cities2.txt
Ranking 0:
SIZE: 4
[700, Santander, 0]
[500, Burgos, 0]
[300, Segovia, 0]
[100, Madrid, 0]
Ranking 1:
SIZE: 3
[600, Palencia, 0]
[400, Valladolid, 0]
[200, Toledo, 0]
Joint Ranking:
SIZE: 7
[100, Madrid, 0]
[200, Toledo, 0]
[300, Segovia, 0]
[400, Valladolid, 0]
[500, Burgos, 0]
[600, Palencia, 0]
[700, Santander, 0]
```

Si ya se hubiera realizado la modificación de Vertex del ejercicio 2, entonces la salida tendría que ser como sigue:

```
$ ./p2_e1b cities1.txt cities2.txt
Ranking 0:
SIZE: 4
[700, Santander, 0, 0]
[500, Burgos, 0, 0]
[300, Segovia, 0, 0]
[100, Madrid, 0, 0]
Ranking 1:
SIZE: 3
[600, Palencia, 0, 0]
[400, Valladolid, 0, 0]
[200, Toledo, 0, 0]
```

```
Joint Ranking:
SIZE: 7
[100, Madrid, 0, 0]
[200, Toledo, 0, 0]
[300, Segovia, 0, 0]
[400, Valladolid, 0, 0]
[500, Burgos, 0, 0]
[600, Palencia, 0, 0]
[700, Santander, 0, 0]
```

El programa utilizará la biblioteca estática `libstack.a` para gestionar la pila. En el *Anexo 1* puedes encontrar cómo crear y usar librerías estáticas en tu proyecto. El programa deberá gestionar correctamente la memoria. La función deberá usar llamadas a `stack_print` para mostrar el contenido de las pilas. Puedes encontrar información adicional sobre el uso de esta función en el *Anexo 2*. Utiliza `valgrind` para verificar que el programa no tiene fugas de memoria.

Ejercicio 2.

En este ejercicio vamos a implementar el algoritmo de búsqueda en profundidad en un grafo utilizando el TAD Pila. El pseudocódigo del algoritmo lo puedes ver a continuación:

Algorithm 2: DFS

```
input  : Graph: G
        Vertex:  $v_f$ , From vertex
        Vertex:  $v_t$ , To vertex
output: Status, st
1 set all vertex Label as WHITE
2 st = OK
3 s = stack_init ()
4 vertex_setLabel ( $v_f$ , BLACK)
5 stack_push (s,  $v_f$ )
6 while stack_isEmpty (s)  $\equiv$  False & st  $\equiv$  OK do
7      $v_o$  = stack_pop (s)
8     if  $v_o \equiv v_t$  then
9         st = END
10    else
11        forall  $v \in \text{AdjacentEdges} (v_o)$  do
12            if vertex_getLabel ( $v$ )  $\equiv$  WHITE then
13                vertex_setLabel ( $v$ , BLACK)
14                stack_push (s,  $v$ )
15            end
16        end
17    end
18 end
19 stack_free (s)
20 return st
```

Para ello, vamos a utilizar la biblioteca del TAD Pila **libstack_fDoble.a** y los TAD Graph y Vertex que has implementado en la práctica anterior.

Para hacer un uso más eficiente de los recursos, vamos a introducir un cambio sobre el TAD Vertex: de cara a mejorar las búsquedas de los vértices dentro del grafo, se añadirá un nuevo atributo al vértice que indica la posición que ocupa en el array de vértices de la estructura de datos del grafo. De esta forma la estructura de datos **_Vertex** definida en el fichero **vertex.c** será:

```
/* START [_Vertex] */
struct _Vertex {
    long id;
    char tag[TAG_LENGTH];
    Label state;

    int index;
};
/* END [_Vertex] */
```


Este cambio permitirá que el algoritmo anterior sea más eficiente (*¿Por qué?*). Sin embargo, deberás incorporar las funciones adecuadas en el **TAD Vertex** para *manejar* este nuevo atributo y actualizar el resto de funciones, tanto las de la interfaz pública como las privadas, para que lo tengan en cuenta.

En este mismo sentido, deberás realizar los cambios oportunos en la implementación del **TAD Graph** para añadir la posición que ocupa cada vertex cuando se inserte en el grafo y actualizar aquellas funciones de la interfaz pública que sean necesarias.

Igualmente deberás añadir las funciones privadas en **graph.c** que consideres oportunas.

Se recomiendan al menos las siguientes funciones privadas (recuerda que deberán ser privadas aquellas funciones que o bien sus *parámetros de entrada* o bien su *retorno* utilicen información relativa a la Estructura de datos utilizada para implementar el Grafo):

```
/* START [Private_functions] */
int _graph_findVertexByTag(const Graph *, char *tag);

Status _graph_insertEdgeFromIndices(Graph *g, const long origIx, const
long destIx);
int _graph_getNumConnections(const Graph *g, int ix);

long *_graph_getConnections(const Graph *g, int ix);
void _graph_setVertexState (Graph *g, Label l);
/* END [Private_functions] */
```

Codificación del algoritmo

Codifica el algoritmo de búsqueda en profundidad mediante la función

```
/**
 * @brief: Makes a search from the origin vertex to the destination vertex
 * of a graph using the depth-first search algorithm
 * using an ADT Stack
 *
 * The function prints each visited vertex while traversing the graph
 *
 * @param g, Pointer to graph
 * @param from_id, id of the origin Vertex
 * @param to_id, id of the destination Vertex
 * @return The function returns OK or ERROR
 */

Status graph_depthSearch (Graph *g, long from_id, long to_id)
```

La función deberá incluir el oportuno control de errores.

Comprobación de la corrección de la función

Crea un programa **p2_e2.c** que utilice la función **graph_depthSearch** del TAD Graph para recorrer un grafo contenido en el fichero pasado como argumento, desde el vértice con de origen (id pasada como argumento) al vértice destino (id pasada como argumento)

En el siguiente ejemplo se muestra una ejecución del programa, en el que se recorre el grafo contenido en g2.txt desde el vertex 100, Madrid al vertex 700, Santander:

```
$ ./p2_e2 g2.txt 100 700
Input graph:
[100, Madrid, 0, 0]: [200, Toledo, 0, 1] [300, Segovia, 0, 2]
[200, Toledo, 0, 1]:
[300, Segovia, 0, 2]: [400, Valladolid, 0, 3] [500, Burgos, 0, 4]
[400, Valladolid, 0, 3]: [600, Palencia, 0, 5]
[500, Burgos, 0, 4]: [600, Palencia, 0, 5] [700, Santander, 0, 6]
[600, Palencia, 0, 5]: [700, Santander, 0, 6]
[700, Santander, 0, 6]:
From Vertex id: 100
To Vertex id: 700
Output:
[100, Madrid, 1, 0]
[300, Segovia, 1, 2]
[500, Burgos, 1, 4]
[700, Santander, 1, 6]
```

El programa utilizará la biblioteca estática **libstack.a** para gestionar una pila de Vertex. El programa deberá gestionar correctamente la memoria. Utilice **valgrind** para verificar que no tiene fugas de memoria.

Ejercicio 3.

En este ejercicio se desea implementar el TAD Stack como un array dinámico de elementos (genéricos), en un fichero `stack.c`. La interfaz pública del TAD la puedes encontrar en el fichero `stack.h`.

El tipo de dato a guardar será un array dinámico de punteros void, de forma que se puedan almacenar, gestionar e imprimir elementos de cualquier tipo

```
#define INIT_CAPACITY 2 // init stack capacity
#define FCT_CAPACITY 2 // multiply the stack capacity

struct _Stack {
    void **item; /*!<Static array of elements*/
    int top; /*!<index of the top element in the stack*/
    int capacity; /*!<xcapacity of the stack*/
};
```

Una pila se inicializará con un tamaño `INIT_CAPACITY` e incrementará su tamaño según el factor `FCT_CAPACITY` cuando esté llena.

Debes implementar tanto la interfaz pública, como las funciones privadas que consideres necesarias.

Debes probar todos tus ejercicios con tu implementación del TAD Stack.

Ayuda:

La función `realloc` permite modificar el tamaño de un espacio de memoria reservado dinámicamente (con `malloc`, `calloc` o `realloc`). En este ejercicio se reservará memoria inicialmente para el número de elementos indicado por `INIT_CAPACITY` y su tamaño se irá incrementando cuando se quieran añadir nuevos elementos a una pila que ya esté llena. Para tener control de la capacidad se ha añadido el atributo `capacity` a la estructura `struct _Stack`. Esto se hace de esta forma para abordar uno de los problemas que surgen en la reserva de memoria dinámica: cuánto se debe incrementar la capacidad de un contenedor, de forma que no sobre mucho espacio pero tampoco se invoque a la función `realloc` continuamente (lo que sería poco eficiente).

APÉNDICES

Apéndice 1.

Punteros a función

Introducción

El lenguaje C permite manejar un tipo de dato, puntero a función, que ofrece un gran potencial, ya que podremos usar funciones de manera dinámica y automatizada, es decir, que dependiendo de las necesidades de nuestro programa, podremos hacer llamadas a unas funciones u otras, dependiendo de lo indicado en esa variable. Es decir, al igual que tenemos punteros que apuntan a variables (y pueden cambiar durante el tiempo de ejecución), podemos hacer llamadas a una función, la cual viene indicada por un puntero.

Como ejemplo, podemos ver la función `stack_print` que usa las funciones definidas en `file_utils.h`. En esta biblioteca, hemos definido nuevos tipos de dato (que no son más que punteros a los tipos que ya conocemos, int, float, etc.) y una serie de funciones para manejarlos (init, copy, compare, print, etc.). Si usamos este tipo de datos, podemos hacer que nuestros programas sean genéricos y, con solo cambiar a la función que se apunta (dependiendo del tipo de dato), el programa funcione igualmente.

La función `stack_print` tiene el siguiente prototipo:

```
/**
 * @brief This function writes the elements of the stack to the
stream.
 * @param fp A pointer to the stream
 * @param s A pointer to the element to the stack
 * @param A pointer to the elements' print function
 * @return Upon successful return, these function returns the number
of characters writted.
 * The function returns a negative value if there was a problem
writing to the file.
 * */
int stack_print(FILE* fp, const Stack *s, P_stack_ele_print f);
```

Es decir, recibe el fichero donde se vaya a imprimir (`stdout`, fichero, etc.), un puntero a la pila a imprimir y un puntero a la función de impresión del tipo de elemento almacenado en la pila. Este último argumento, viene definido previamente en el archivo de cabecera del TAD pila:

```
/**
 * @brief Typedef for a function pointer to print a stack element at
stream
 */
typedef int (*P_stack_ele_print)(FILE *, const void*);
```

Y nos indica que es una función que devuelve un `int`, recibe un puntero a fichero (de nuevo, `stdout`, fichero, etc.) y un puntero a void. Este último argumento es muy importante, ya que el hecho que sea `void *` nos va a permitir que podamos pasar diferentes tipos de datos, como hemos venido haciendo hasta ahora.

Todas las funciones que se pasen como argumento en `stack_print` deben cumplir con este prototipo. Así, en `file_utils.h`, como puedes ver, todas las funciones de impresión mantienen los mismos argumentos, a pesar de manejar tipos de dato diferentes.

Por ejemplo, si nuestra pila (llamada `s1`) contuviese elementos de tipo:

`int` la llamada a la función para imprimir la pila sería:

```
stack_print(stdout, s1, int_print);
```

`float` la llamada a la función para imprimir la pila sería:

```
stack_print(stdout, s1, float_print);
```

Apéndice 2

Creación y uso de bibliotecas estáticas

Introducción

Las bibliotecas son una forma sencilla y versátil de modularizar y reutilizar código. Una biblioteca es un archivo que contiene varios ficheros objetos (**.o**) y que, por tanto, puede ser usado como un único programa en la fase de enlazado con otros programas.

Los sistemas **UNIX** permiten crear dos tipos diferentes de bibliotecas, las bibliotecas estáticas y las bibliotecas dinámicas. En este documento se aborda el proceso de creación, compilación y uso de una biblioteca estática.

Creación de una biblioteca estática

Una biblioteca estática consta de:

- Un conjunto de ficheros objetos (**.o**) empaquetados en un único archivo de *filetype* **.a**.
- Un conjunto de ficheros (**.h**) con la interfaz de la biblioteca.

Para crear una biblioteca estática debemos:

- Compilar cada uno de los objetos que forman la biblioteca
- Empaquetar los **.o** en un único archivo por medio la utilidad **ar**

Por convenio, los nombres de todas las bibliotecas estáticas comienzan por **lib** y tienen **.a** por extensión.

Ejemplo:

Supongamos que disponemos de los ficheros **stack_fp.c** y **stack_types.c** que contienen el código de las funciones que deseamos incluir en la biblioteca **libstack_fp.a**. Para obtener los ficheros objeto compilamos los ficheros fuente:

```
$ gcc -c stack_fp.c
$ gcc -c stack_types.c
```

A continuación empaquetamos los ficheros obtenidos **.o** con **ar** (un empaquetador similar a **tar**).

```
$ ar rcs libstack_fp.a stack_fp.o stack_types.o
```

El significado de las opciones que se le dan a **ar** es el siguiente:

Flag	Significado
r	Reemplaza los ficheros si ya existían en el paquete.

Flag	Significado
c	Crea el paquete si no existe.
s	Construye un índice del contenido.
t	Lista el contenido de un paquete (o biblioteca).
x	Extrae un fichero de un paquete (o biblioteca).

Uso de una biblioteca desde una terminal

Para utilizar las funciones de la biblioteca estática en una aplicación (por ejemplo en un programa `main.c`) es necesario enlazar las funciones incluidas en la biblioteca cuando creamos el ejecutable de la aplicación. Para ello debemos indicar al compilador qué biblioteca queremos utilizar y el lugar donde se halla.

Ejemplo:

Supongamos que hemos escrito un programa `main.c` que hace uso de las funciones incluidas en la biblioteca `libstack_fp.a` creada en el apartado anterior. Para crear el ejecutable `main` debemos:

- Compilar el programa `main.c`:

```
$ gcc -c main.c
```

Notad que al incluir el fichero `main.c` llamadas a las funciones cuyos prototipos están declarados en los ficheros `.h` de la interfaz de la biblioteca (p.e., `#include "stack_fp.h"`), estos deberían estar un directorio incluido en el `path` del compilador. De no ser así debe indicarse al compilador dónde se encuentran los ficheros de la interfaz mediante la opción `-I`:

```
$ gcc -c main.c -Idir_lib
```

donde `dir_lib` es el directorio donde hemos guardado los ficheros `.h` de la biblioteca.

- Enlazar el programa con la biblioteca indicando donde está (`dir_lib`) y cuál es su nombre (`stack_fp`):

```
$ gcc -o main main.o -Ldir_lib -lstack_fp
```

Consideraciones:

En el ejemplo se supone que la biblioteca y su fichero de interfaz se encuentran en un directorio llamado `dir_lib`.

- La opción `-I` se usa para indicar donde se encuentran los ficheros de la interfaz de la biblioteca (en este caso, `stack_fp.h` y `stack_types.h`).

- La opción `-L` indica el directorio donde se encuentra la biblioteca (en este caso `libstack_fp.a`).
- La opción `-l` indica los nombres de las bibliotecas que se van a usar. Sin embargo los nombres de las bibliotecas no deben escribirse ni con el prefijo `lib` ni con la extensión `.a` ya que el compilador espera que se sigan las normas de nombrado anteriormente citadas.

Creación y uso de una biblioteca con Makefile

A continuación se muestra un fichero `makefile` que permite automatizar todo el proceso, tanto la creación de la biblioteca como del programa. En este caso se supone que todos los ficheros fuentes se encuentran en el directorio actual.

```
CC=gcc
CFLAGS=-Wall -ggdb
IFLAGS=-I./
LDLFLAGS=-L./
LDLIBS=-lstack_fp
### -lm enlaza la biblioteca matematica, -pthread enlaza la biblioteca de
hilos
LIBS = -lm -pthread

all: libstack_fp.a main

#####
# $@ es el item que aparece a la izquierda de ':'
# $< es el primer item en la lista de dependencias
# $^ son todos los archivos que se encuentran a la derecha de ':'
(dependencias)
#####

main: main.o libstack_fp.a
    $(CC) -o $@ $< $(LDLFLAGS) $(LDLIBS) $(LIBS)

main.o: main.c
    $(CC) -c -o $@ $< $(CFLAGS) $(IFLAGS)

#####
##### Crea la biblioteca con las fuentes: stack_fp.c stack_types.c
#####

stack_fp.o: stack_fp.c stack_fp.h stack_types.h
    $(CC) -c -o $@ $< $(CFLAGS)

stack_types.o: stack_types.c stack_types.h
    $(CC) -c -o $@ $< $(CFLAGS)

libstack_fp.a: stack_fp.o stack_types.o
    $(AR) rcs $@ $^

libs: libstack_fp.a
```



```
clean:
    rm -f *.o *.a
```

Si ya disponemos del fichero de la biblioteca (`libstack_fp.a`) y unicamente queremos crear el ejecutable, nuestro `Makefile` se reduciría a las siguientes sentencias:

```
CC=gcc
CFLAGS=-Wall -ggdb
IFLAGS=-I./
LDFLAGS=-L./
LDLIBS=-lstack_fp
# -lm enlaza la biblioteca matematica, -pthread enlaza la biblioteca de
# hilos
LIBS = -lm -pthread

all: main

#####
# $@ es el item que aparece a la izquierda de ':'
# $< es el primer item en la lista de dependencias
# $^ son todos los archivos que se encuentran a la derecha de ':'
# (dependencias)
#####

main: main.o libstack_fp.a
    $(CC) -o $@ $< $(LDFLAGS) $(LDLIBS) $(LIBS)

main.o: main.c
    $(CC) -c -o $@ $< $(CFLAGS) $(IFLAGS)
```